Verificación y Validación con Herramientas

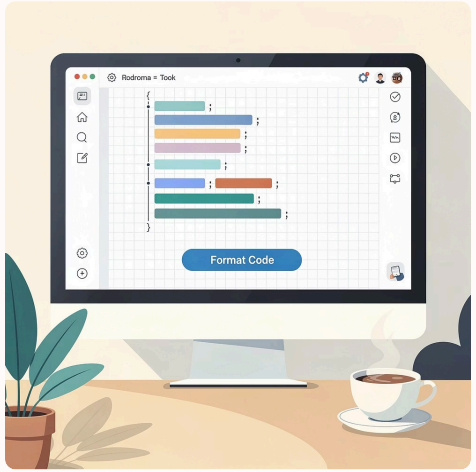
UNIDAD 3 · SEMANA 3

Automatización de calidad, análisis estático, pre-commit hooks y gobernanza del repositorio para certificar software listo para integración.



Herramientas de Automatización de Calidad

La automatización de calidad consiste en delegar en herramientas la auditoría del código fuente, eliminando la subjetividad humana y asegurando que ningún incremento viole las reglas técnicas del equipo.



Formatter

Ajusta automáticamente el estilo visual del código: sangrías, espaciados, comillas y saltos de línea. Elimina discusiones superficiales en revisiones por pares. Ejemplos: **Black** (Python) y **Prettier** (JavaScript/TypeScript).



Linter

Analiza el árbol de sintaxis abstracta para detectar errores de lógica, variables no utilizadas, llamadas a funciones inexistentes y patrones inseguros antes de ejecutar el programa.

¿Qué Detecta un Linter?

Variables e Importaciones No Utilizadas

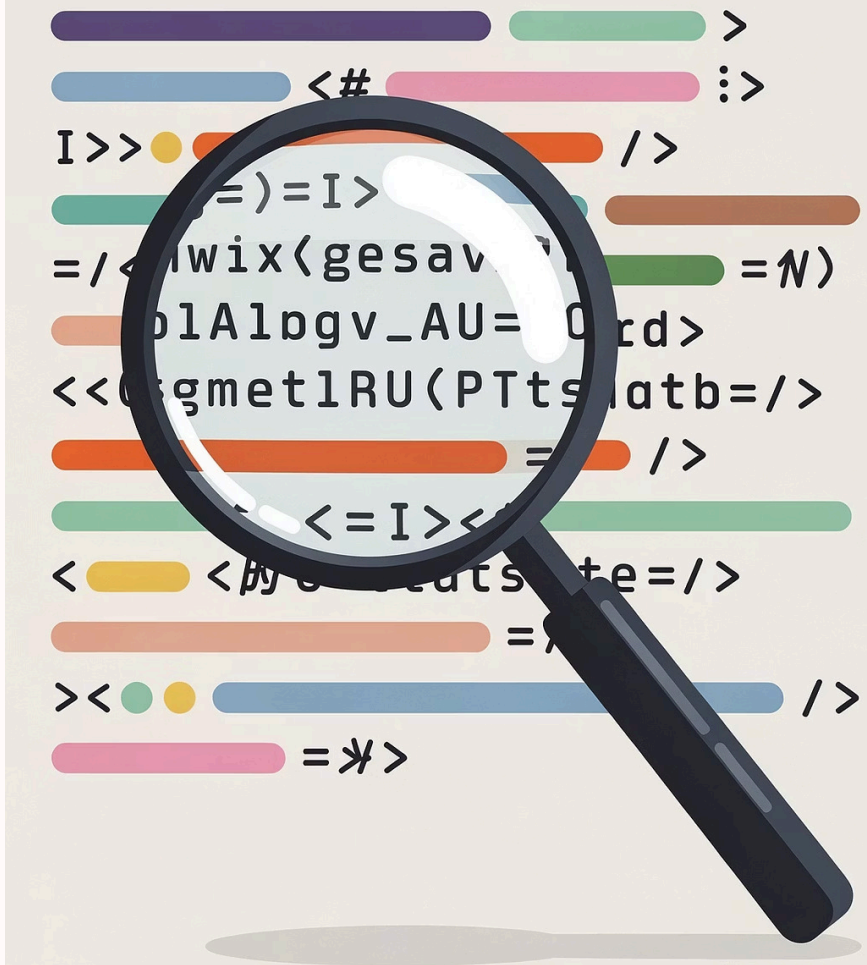
Reduce el desperdicio de memoria y optimiza el empaquetado del software.

Errores de Sintaxis Tempranos

Alerta sobre llamadas a funciones o interfaces inexistentes antes de la ejecución.

Malas Prácticas y Patrones Inseguros

Detecta uso desmedido de variables globales o mutaciones peligrosas que vulneran las guías oficiales del lenguaje.



Auditoría Avanzada en Frío

Análisis Estático con SonarQube

Examina la salud estructural de la aplicación **sin ejecutar el software**. Calcula métricas clave:

- **Complejidad Ciclomática:** Mide caminos lógicos independientes en una función. Un índice elevado incrementa la probabilidad de fallos.
- **Duplicación de Código:** Identifica bloques redundantes, forzando el principio DRY.
- **Vulnerabilidades y Bugs Latentes:** Detecta debilidades que comprometen la seguridad o provocan fallos en alta concurrencia.

Seguridad de Dependencias

Las aplicaciones modernas integran numerosas bibliotecas de terceros. Las herramientas de seguridad escanean archivos de manifiesto (`package.json`, `requirements.txt`) y contrastan versiones contra bases de datos globales de vulnerabilidades conocidas (**registros CVE**).

Al detectar un paquete inseguro, el sistema bloquea el flujo y emite un reporte técnico con el nivel de riesgo y la versión parchada recomendada.

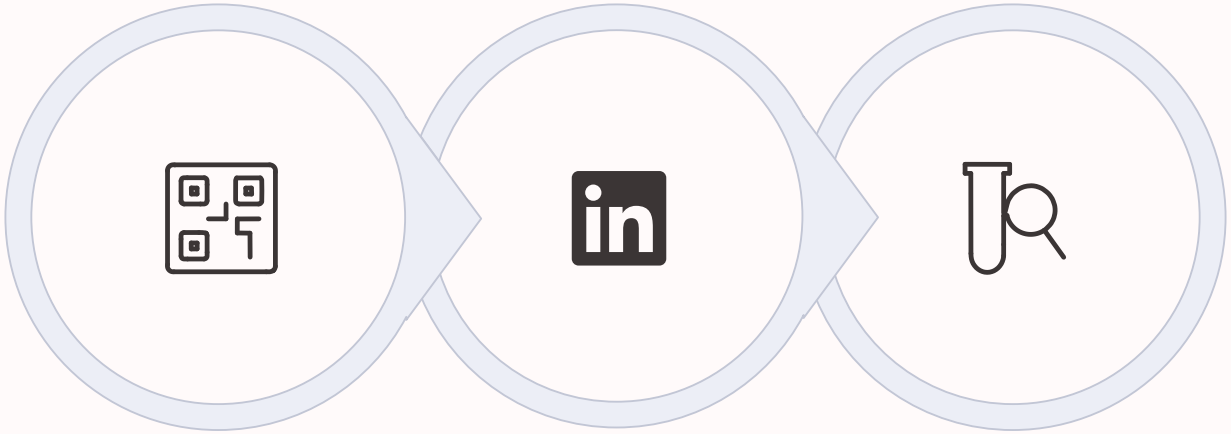
Pre-Commit Hooks: Blindaje de Calidad

El control de calidad es ineficiente si depende de la memoria del programador. Los **Pre-commit Hooks** garantizan que ningún desarrollador pueda guardar código que viole la Definition of Done.

> Un **Hook de Git** es un script automatizado que se dispara ante eventos específicos del control de versiones. El pre-commit hook se ejecuta **inmediatamente después de invocar** `git commit`, pero antes de que el mensaje sea consolidado en el historial.

Flujo de Validación del Pre-Commit Hook

Al activar el hook, se ejecuta secuencialmente la siguiente cadena de custodia sobre los archivos modificados:



Si cualquier paso falla — el formateador detecta errores, el linter encuentra variables no declaradas, o un solo test queda en rojo — el hook **aborta el commit** y devuelve un código de error. El código defectuoso queda retenido en el espacio de trabajo local hasta sanear la deuda técnica.

Gobernanza del Repositorio Local

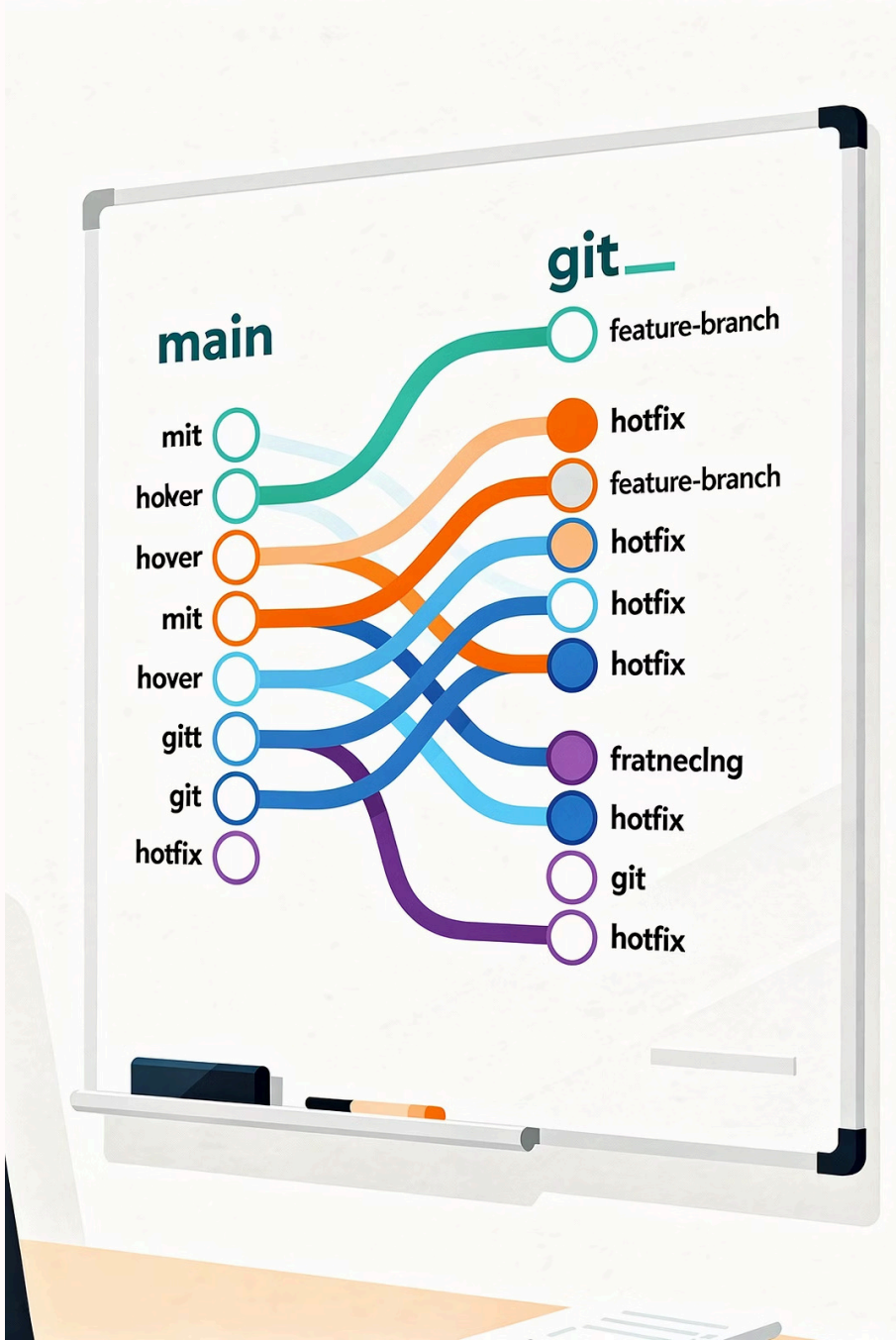
La gestión del ciclo de vida del software requiere flujos de trabajo organizados que garanticen que cada cambio sea auditable, transparente y trazable.

Feature Branches y Forks

Todo incremento o corrección nace en una **rama independiente**. El nombre técnico debe reflejar la incidencia o tarea, manteniendo aislado el entorno de desarrollo.

Pull Requests (PR)

Propuesta formal para fusionar código en la rama principal. Es una **interfaz de gobernanza clínica** donde se consolida el proceso de calidad.



Pull Requests: Revisión por Pares Trazable

Trazabilidad Automática

El PR debe enlazarse explícitamente con el Issue de origen mediante palabras clave, asegurando que cada cambio responda a un requerimiento o bug reportado.

Auditoría de Cambios

Permite a los revisores examinar el **diff técnico**: la comparación línea por línea del código agregado, modificado o eliminado.

Code Review Documentado

Comentarios técnicos, debates de diseño y la aprobación final quedan grabados de forma **permanente y cronológica** como evidencia histórica de calidad.

El Informe de Calidad en Verde

El artefacto consolidado que certifica que el software cumple con la **Definition of Done** y está listo para ser integrado. Se compone de cuatro evidencias obligatorias:

1

Gobernanza de Entrada

Enlace trazable entre el Pull Request y el Issue estructurado que originó la tarea.

2

Evidencia Sintáctica y Estética

Logs del pre-commit hook que prueban la ejecución exitosa del Formatter y Linter sin excepciones.

3

Evidencia Dinámica

Reporte de Cobertura de Código (Code Coverage) en verde, certificando el umbral mínimo de pruebas.

4

Evidencia Estructural

Diagnóstico de SonarQube con deuda técnica aceptable, complejidad ciclomática controlada y dependencias sin vulnerabilidades críticas.

Resumen: La Cadena de Calidad Completa

Herramientas
Formatter y Linter locales

Pre-Commit
Validación automática al confirmar

Gobernanza
Branches y Pull Requests

Informe
Evidencias consolidadas en verde

Dominar esta cadena de automatización transforma al desarrollador en un profesional capaz de certificar la calidad de su software con evidencias objetivas, trazables y reproducibles — el estándar de la ingeniería de software moderna.

